Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

y Ciencias de la Computación

Ing. Geovani de León

Programación II

INFORME SISTEMA DE GESTION PARA ALQUILER DE VEHICULOS

ALUMNOS

Denis Leonel Gonzalez Aragon

Jimmy Lizandro Ulpino Tiul Xol

Larsson Rafael Buc Sierra

Cobán, Alta Verapaz, Septiembre del 2026.

Índice

1. Introducción 3
2. Problema abordado 4
3. Objetivos 5
   Objetivo general 5
   Objetivos específicos 5
4. Diagrama de clases (UML) 6
5. Diseño de base de datos (Modelo Entidad-Relación) 8
6. Patrones de diseño aplicados 10
7. Resultados obtenidos 11
8. Conclusiones 11

1. Introducción

El presente proyecto contempla el diseño y la implementación de un Sistema de Gestión de Alquiler de Autos, una aplicación orientada a objetos desarrollada en Java (Swing) que guarda su información en una base de datos relacional en PostgreSQL. El sistema centraliza la administración de la flota de vehículos, las sucursales, los clientes y empleados, el mantenimiento de las unidades, los contratos de alquiler y la emisión de facturas.

Este documento presenta la estructura del software a partir del modelo de dominio representado en el diagrama de clases UML, y la forma en que ese modelo se traduce en el diseño de la base de datos. El sistema se organiza en tres capas: presentación (vista), lógica de negocio (logica) y persistencia de datos (conexion), lo que facilita su mantenimiento y su crecimiento a futuro.

2. Problema abordado

Las empresas de alquiler de autos que trabajan con procesos manuales, hojas de cálculo o registros en papel suelen enfrentar problemas que afectan tanto sus ingresos como la atención al cliente.

Inconsistencia en el estado de la flota. Sin un control en tiempo real, no se sabe con certeza qué vehículos están disponibles, en mantenimiento o en qué sucursal se encuentran, lo que provoca que se asignen autos que no están aptos para alquilar.

Descontrol transaccional e inseguridad de datos. No queda registro de qué empleado procesó cada contrato, y la información sensible de los clientes, como la licencia de conducir, el DPI y el NIT, queda expuesta a pérdidas o accesos indebidos.

Errores en el cálculo de cargos. Calcular a mano las tarifas diarias, los recargos por mora o los servicios adicionales produce cobros incorrectos.

Desconexión entre contratos y facturas. Cuando los contratos y la facturación se llevan por separado, los reportes operativos no coinciden con lo realmente cobrado.

El sistema responde a estos problemas automatizando todo el ciclo de vida del alquiler, con validaciones en tiempo real y reglas de integridad referencial en la base de datos.

3. Objetivos

Objetivo general

Desarrollar un sistema de gestión de alquiler de autos en Java Swing y PostgreSQL, aplicando el paradigma orientado a objetos y una arquitectura en capas, que permita administrar de forma eficiente y segura la flota, los clientes, los contratos y la facturación, reduciendo los errores del control manual y mejorando la atención al cliente.

Objetivos específicos

1. Modelar el dominio del negocio mediante un diagrama de clases UML que defina los atributos, métodos y relaciones entre vehículos, clientes, empleados, contratos, mantenimientos y facturas.
2. Implementar una interfaz gráfica clara e intuitiva en Java Swing, con un diseño de tema oscuro y recursos (imágenes e íconos) organizados bajo la estructura estándar de Maven, para que el programa funcione en cualquier equipo.
3. Asegurar la autenticación y el control de acceso, validando el usuario y la clave de cada empleado contra la base de datos de PostgreSQL y restringiendo las operaciones según su perfil.
4. Automatizar el cálculo del costo de cada alquiler, considerando los días consumidos y los cargos adicionales, y generar de inmediato la factura correspondiente.

4. Diagrama de clases (UML)

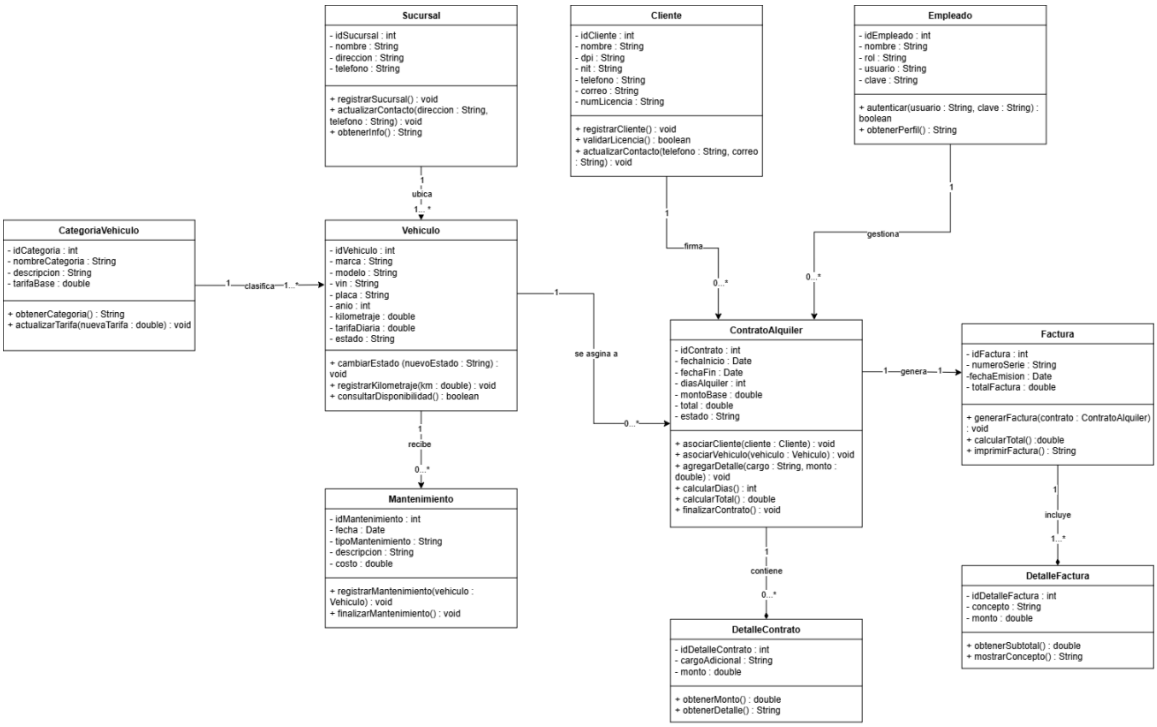
El modelo de clases del sistema agrupa las responsabilidades del negocio en diez clases, organizadas según su función.

Flota y operación: Sucursal, Vehículo, CategoriaVehiculo y Mantenimiento. Representan los autos, el lugar donde se encuentran, su segmento (sedán, SUV, pickup) y las intervenciones técnicas que reciben.

Actores del sistema: Cliente y Empleado. El cliente es quien alquila y el empleado es quien procesa el contrato y accede al sistema con su usuario y clave.

Transacciones y contabilidad: ContratoAlquiler, DetalleContrato, Factura y DetalleFactura. Registran el alquiler, sus cargos adicionales y el cobro formal.

Las relaciones principales son las siguientes: una sucursal y una categoría tienen muchos vehículos; un vehículo tiene muchos mantenimientos; un cliente, un empleado y un vehículo pueden estar en muchos contratos; un contrato tiene muchos detalles y una sola factura; y una factura tiene muchos detalles.



El diagrama muestra los atributos y métodos de cada clase, la multiplicidad de cada relación y la composición (rombo relleno) entre el contrato y sus detalles, y entre la factura y sus detalles.

Clase	Atributos principales
Sucursal	idSucursal, nombre, direccion, telefono
CategoriaVehiculo	idCategoria, nombreCategoria, descripcion, tarifaBase
Vehiculo	idVehiculo, marca, modelo, vin, placa, anio, kilometraje, tarifaDiaria, estado
Mantenimiento	idMantenimiento, fecha, tipoMantenimiento, descripcion, costo
Cliente	idCliente, nombre, dpi, nit, telefono, correo, numLicencia
Empleado	idEmpleado, nombre, rol, usuario, clave
ContratoAlquiler	idContrato, fechaInicio, fechaFin, diasAlquiler, montoBase, total, estado
DetalleContrato	idDetalleContrato, cargoAdicional, monto
Factura	idFactura, numeroSerie, fechaEmision, totalFactura
DetalleFactura	idDetalleFactura, concepto, monto

Justificación de relaciones. La siguiente tabla explica la multiplicidad y el tipo de cada relación del diagrama.

Relación	Multiplicidad	Tipo	Justificación
Sucursal ubica Vehiculo	1 a 1..*	Asociación	Una sucursal alberga uno o varios vehículos, y cada vehículo pertenece a una sola sucursal base.
CategoriaVehiculo clasifica Vehiculo	1 a 1..*	Asociación	Una categoría agrupa varios vehículos con características similares, y todo vehículo pertenece a una categoría.
Vehiculo recibe Mantenimiento	1 a 0..*	Asociación	Un vehículo puede tener desde ningún servicio técnico hasta muchos, y cada mantenimiento se aplica a un vehículo específico.
Cliente firma ContratoAlquiler	1 a 0..*	Asociación	Un cliente puede firmar ningún contrato o varios, y cada contrato

Relación	Multiplicidad	Tipo	Justificación
			pertenece a un solo cliente.
Empleado gestiona ContratoAlquiler	1 a 0..*	Asociación	Un empleado procesa varios contratos, y cada contrato registra al empleado responsable de emitirlo.
Vehiculo se asigna a ContratoAlquiler	1 a 0..*	Asociación	Un vehículo se asigna a un contrato durante el alquiler y, con el tiempo, acumula varios contratos.
ContratoAlquiler contiene DetalleContrato	1 a 0..*	Composición	Los cargos adicionales no pueden existir sin el contrato al que pertenecen.
ContratoAlquiler genera Factura	1 a 1	Asociación	Un contrato liquidado da origen a una única factura.
Factura incluye DetalleFactura	1 a 1..*	Composición	Los renglones impresos dependen de la factura que los contiene y no existen por separado.

5. Diseño de base de datos (Modelo Entidad-Relación)

La base de datos en PostgreSQL, administrada con pgAdmin 4, se construyó a partir del diagrama UML. La integridad referencial se garantiza con claves primarias (PK) y claves foráneas (FK) en cada relación del negocio.

Tabla	Claves	Descripción
Sucursal	idSucursal (PK)	Sede donde se encuentra físicamente cada vehículo.
CategoriaVehiculo	idCategoria (PK)	Catálogo de segmentos con su tarifa base (sedán, SUV, pickup).
Vehiculo	idVehiculo (PK), idSucursal (FK), idCategoria (FK)	Unidades con su VIN, placa, estado (disponible, alquilado, en mantenimiento) y kilometraje.
Mantenimiento	idMantenimiento (PK), idVehiculo (FK)	Intervenciones técnicas realizadas a cada vehículo.

Tabla	Claves	Descripción
Cliente	idCliente (PK)	Datos de contacto, DPI, NIT y número de licencia.
Empleado	idEmpleado (PK)	Personal autorizado, con su usuario y clave de acceso.
ContratoAlquiler	idContrato (PK), idCliente (FK), idEmpleado (FK), idVehiculo (FK)	Vincula al cliente, al empleado y al vehículo en un periodo de alquiler, con los montos desglosados.
DetalleContrato	idDetalleContrato (PK), idContrato (FK)	Cargos adicionales asignados a un contrato.
Factura	idFactura (PK), idContrato (FK, relación 1:1)	Comprobante del cobro de cada contrato.
DetalleFactura	idDetalleFactura (PK), idFactura (FK)	Renglones individuales de cada factura.



6. Patrones de diseño aplicados

Conforme a los lineamientos del curso, se aplicaron dos patrones de diseño.

A. Patrón Singleton (conexión a la base de datos). Se aplicó en la clase conexion.ConexionBD, que administra el enlace con PostgreSQL. Evita abrir varias conexiones simultáneas e innecesarias, que saturarían los recursos del sistema. El

Singleton garantiza una única instancia de conexión activa durante toda la ejecución y centraliza el acceso a ella de forma segura y eficiente.

B. Patrón MVC / arquitectura en capas (estructura del software). Se aplicó al organizar el código en paquetes: vista (las interfaces en Java Swing, como login.java), logica (los gestores de negocio) y las clases de persistencia. Resuelve el acoplamiento rígido entre la interfaz y la lógica, de modo que un cambio visual en las pantallas no afecta las reglas de negocio ni las consultas a la base de datos, y se facilita el mantenimiento y el control de versiones en equipo.

7. Resultados obtenidos

Interfaz gráfica moderna. Se logró un panel en tono oscuro con el logotipo corporativo integrado y las ventanas centradas automáticamente.

Autenticación funcional. El módulo de acceso se conectó a PostgreSQL y valida las credenciales de los empleados.

Recursos organizados. Las imágenes y demás recursos estáticos quedaron bajo el estándar de Maven (src/main/resources/media/), lo que garantiza que el proyecto funcione al clonar el repositorio de GitHub o al generar ejecutables .jar.

Mapeo completo del modelo. Todas las entidades del diagrama UML se tradujeron a clases ejecutables en Java.

8. Conclusiones

La metodología orientada a objetos y el diagrama de clases UML permitieron pasar al código en Java de forma fluida, con menos errores estructurales.

El patrón Singleton mejoró el rendimiento de la aplicación al administrar de forma centralizada la conexión con el servidor de base de datos.

La organización modular bajo MVC y la estructura de Maven hacen que el sistema sea escalable, fácil de mantener y apto para el trabajo colaborativo en equipo.